

CSC 583 Natural Language Processing

Fall 2025

Homework #1: Text Preprocessing

There is no base code is provided for this homework. However, supplemental files are found on the [course Github](#), under "HW1".

1. Specifics

Your task in this assignment is to write a Python program that accepts as input a plain text document and compute some statistics of the content. Specifically your program should output the following:

- a. Total number of paragraphs.
- b. Total number of sentences.
- c. Total number of words/tokens -- AFTER you applied the specified tokenization described below (*).
- d. Total number of unique words/tokens (i.e., "word types" in [J&M section 2.2-2-4](#)).
- e. Ranked frequency counts of word/token types (sorted in the descending order of frequency)

You **write the code from scratch**. You may use Python native string-related functions (e.g. lower()) or a few other libraries for utility purposes (such as re, numpy, pandas), but **NO** NLP-specific libraries.

The only exceptions are these two functions in NLTK:

- **sent_tokenize()** -- for sentence segmentation ([NLTK book, ch3](#), section 3.8)
- **word_tokenize()** -- for (preliminary) word tokenization ([NLTK book, ch3](#), section 3.1)

Other specifics:

1. Normalize all words to **lower case**. But do **not** apply stemming (nor filter out so-called stopwords).
2. (*) Words must be tokenized according to the following steps and rules.

Step 1: As a **preliminary step**, apply NLTK's **word_tokenize()** to some short test text (e.g. first few sentences from the task 1 data below, "[sample 2025.txt](#)") and examine how the tokenizer does tokenization with respect to punctuations (and contractions).

Step 2: Proceed to apply **word_tokenize()** to the whole input text and process tokens to conform to these rules below. Note that, for some rules, you may need to fix up (word_tokenized()) tokens or look around the tokens before/after the one you are processing to do it correctly.

Rules:

- **Punctuations** should be separated but NOT discarded. Also:
 - You only separate punctuations that occurred only at the **leading or trailing** positions of a word. If a punctuation (or punctuations) occur(s) within a word, you do not separate the word. For example, "\$3.19" should be separated into "\$" and "3.19". **If NLTK word_tokenize() didn't adhere to it for any instance, do NOT take NLTK's tokenization and override yourself by separating ONLY the leading and the trailing punctuations.**

- If a consecutive (leading and trailing) punctuations were tokenized as one token (e.g. "...", "--") by NLTK `word_tokenize()`, treat them as one token/word.
 - Or if a consecutive (leading and trailing) punctuations were tokenized as multiple tokens (e.g. "!", "!" from "!!") by NLTK `word_tokenize()`, treat them as multiple tokens (i.e, as is).
 - Also if a word ends with a period (.), you don't have to check if it is a known acronym (e.g. "mr.", "m.p.g.", "e.g.", "etc."). Just use NLTK's tokenization/split.
- **Contractions** must be expanded and converted to the root/lemma tokens. Although some contractions are ambiguous (e.g. "they'd" could be "they would" or "they had"), in this assignment you can make these simplifying assumptions.

NLTK `word_tokenize()` splits contractions at the quote character (') or special cases such as "n't". For example, it splits "they'd" to two tokens 'they' and 'd' , and "don't" to 'do' and 'n't'.

Contractions to convert:

- "n't" -- assume "not" for all instances (e.g. "don't" -> "do" and "not"), EXCEPT for these special cases (where you'll need to look at the preceding token to determine):
 - won't -- "will" and "not"
 - can't -- "can" and "not"
 - shan't -- "shall" and "not"
- "ll" -- assume "will" for all instances; e.g. "they'll" -> "they" and "will"
- " 've" -- assume "have" for all instances; e.g. "they've" -> "they" and "have"
- "'d" -- assume "would" for all instances; e.g. "they'd" -> "they" and "would"
- "'re" -- assume "are" for all instances; e.g. "they're" -> "they" and "are"
- "'s" -- assume possessive (i.e., an *apostrophe-s*); e.g. "phone's" -> "phone" and "'s" ==> thus no change, EXCEPT for these special cases:
 - "let's" -- "let" and "us"
 - obvious contraction of "is" (mostly used with a singular pronoun or a wh-word; e.g. "it's" -> "it" and "is"). In this assignment, apply this rule to these words: "he's", "she's", "it's", "that's", "here's" and "there's", "what's", "when's", "where's", "which's", "who's" and "how's".
- other special cases:
 - i'm -- "i" and "am".
- Note that, if a word contains multiple contractions (e.g. "shouldn't've"), you must **separate ALL of them** (e.g. "should", "not", "have").

3. Read in the input file only once.

2. Applications/Experiments

Apply your code to the following two tasks. Follow the instructions for each task.

1. [Task 1] Apply your code to the file **"sample 2025.txt"**.

- This file is formatted such that a whole sentence is written in a line (however long it is). So one physical line equates to one sentence.
- Sort the word types by their frequency counts, and write them to an output file, named **"output1.txt"**. Words that have the same frequency count should be sorted by the ascending lexicographical order.

- The output file should indicate the requested counts/statistics at the top of the file (before the word type frequencies), for example in the following form.

```
# of paragraphs = 8
# of sentences = 31
# of tokens = 567
# of unique tokens = 274
```

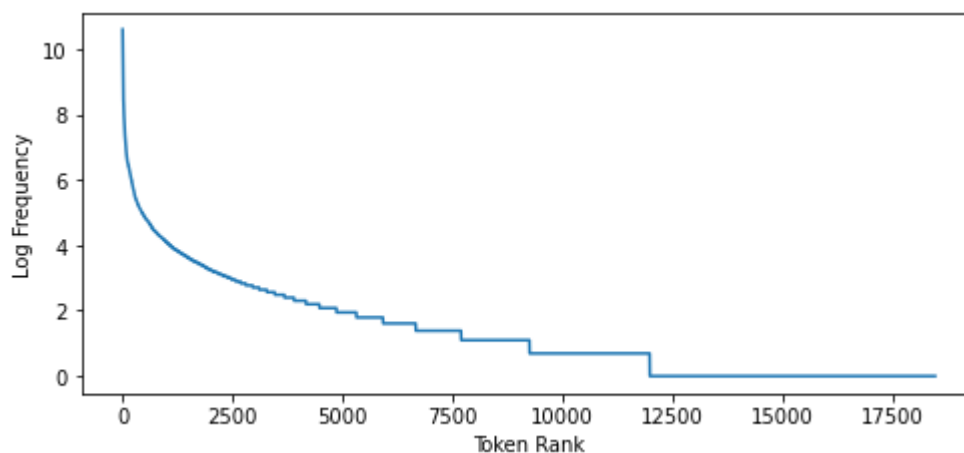
- Also here is the [entire output](#) of the results. Note the left-most column (1-based index) is the *rank* of the token. Format your output in the same way.
- Note that either text cover all cases of punctuations and contractions. You should create your own test input file to ensure the correctness of your implementation.

2. [Task 2] Apply your code to the file ["war-and-peace.txt"](#). This is a text version of the book "War and Peace" by Leo Tolstoy.

- Ensure to open the file in the utf-8 mode (since the file encoding is utf-8).
- This file is formatted in a fixed column-width (so that a line will not have no more than 80 characters). So you will need to think about how to obtain the number of sentences. It's up to you to figure it out. As a hint, you can use NLTK's `sent_tokenize()` function.
- Do the same processing for output as the first task. Show **all words**, sorted by the descending order of the frequency. Name the output file **"output2.txt"**. FYI, the counts/statistics would be something like these:

```
# of paragraphs: 12169
# of sentences: 31911
# of tokens: 672876
# of unique tokens: 18456
```

- FYI, here is an [example output](#) showing the top 500 most frequent tokens.
- Furthermore, create a **chart** that compares the (log of base e) frequencies of the words against their *rank*s, which effectively should show a curve of [Zipf's Law](#). Below is an example chart for war and peace:



- To create the chart, a simple reference is found [here](#) (especially in the section 'Plotting with Python').

3. Deliverables

Submit the following:

1. Source code -- a Python notebook file, plus **its html or pdf file**.
2. Two output files.
3. A write up document (in **docx or pdf**; **minimum 1.0 page**).

Requirements:

- **IMPORTANT:** Your code file must have **your name** at the top of the file (in a comment section).
 - **IMPORTANT:** The write-up document must **your name**, the **course name** (CSC 583) and the **assignment number** (HW#1) at the top of the file (IN the file, of course).
Also names of all collaborators you worked with (if any). This includes GenAI tools such as CoPilot and ChatGPT.
 - The write-up should include:
 - Description of your development environment -- platform (e.g. Google CoLab, MacOS version XX), IDE (e.g. Jupyter Notebook, none/python script), libraries and packages (e.g. re, numpy), etc.
 - The chart generated for Application task #2, and your comments on the distribution of the frequencies.
 - **Reflections and general comments**, including:
 - Whether or not your output matched with the sample output (and if not, why you think the reasons were).
 - What you learned from this assignment.
 - How difficult you felt this assignment was.
 - Any particular difficulties you encountered.
 - How you would do/approach differently next time (if there was one).
 - and anything else.
-

5. Submission

Submit the necessary files on D2L. DO NOT ZIP the files -- SUBMIT EACH FILE SEPERATELY.